



Hour 1 — Components, JSX & Props

Duration: 60 minutes | 10–12 slides

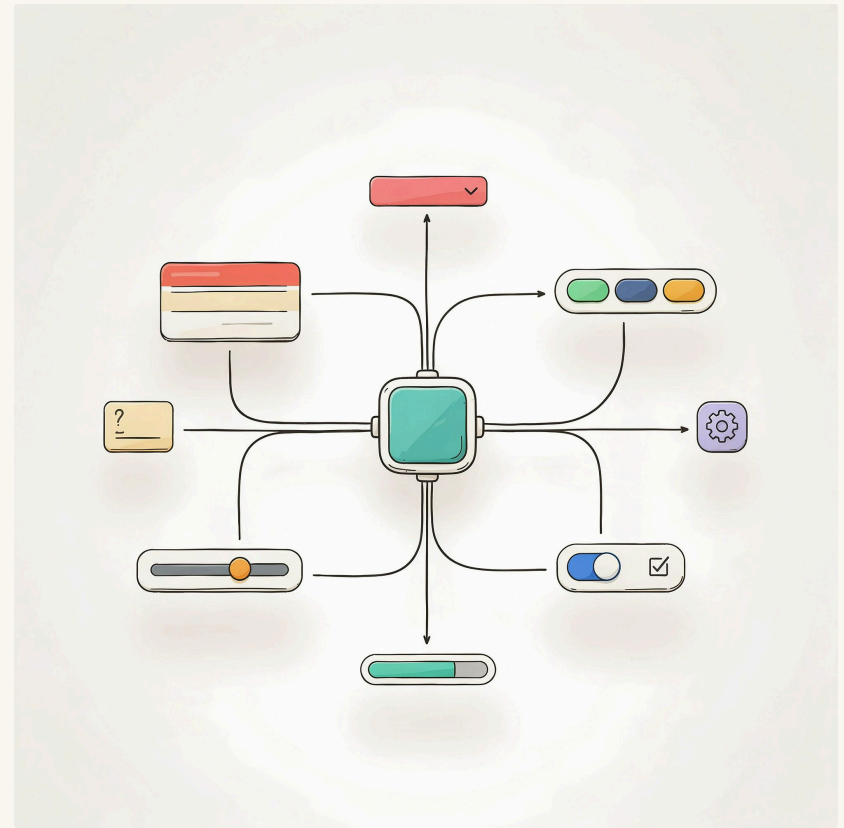
Slide 1 — React Components & JSX

Learning Objectives

- Understand React components
- Write functional components
- Understand JSX syntax
- Pass data using props
- Use children and default prop values
- Build a small component-based UI

Key idea:

React UI = a tree of reusable components.





Slide 2 — What is a React Component?

- A component is a **reusable UI building block**
- Modern React uses **function components**
- A component:
 - Accepts inputs
 - Returns JSX
 - Can be reused multiple times

```
function Welcome() {  
  
  return <h1>Student Portal</h1>;  
  
}
```

Bridge: Think of a component like a Terraform module:

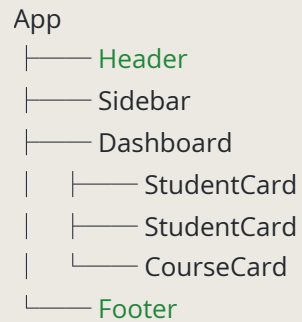
Define once

Reuse multiple times

Provide different inputs

Slide 3 — Component Tree

Example Student Portal:



```
graph TD; App --> Header; App --> Sidebar; App --> Dashboard; App --> Footer; Dashboard --> StudentCard1[StudentCard]; Dashboard --> StudentCard2[StudentCard]; Dashboard --> CourseCard;
```

App

- Header
- Sidebar
- Dashboard
 - StudentCard
 - StudentCard
 - CourseCard
- Footer

Benefits

Reusability

Maintainability

Separation of responsibilities

Easier testing

Slide 4 — JSX: HTML-Like JavaScript

JSX allows UI to be written in a familiar syntax.

```
function Welcome() {  
  
  return (  
  
    <div className="welcome-card">  
  
      <h1>Student Portal</h1>  
  
      <p>Welcome back!</p>  
  
    </div>  
  
  );  
  
}
```

Important rules:

- Use `className`, not `class`
- JavaScript expressions use `{ }`
- Component names use **PascalCase**
- JSX must have one root element or Fragment

Slide 5 — JSX and JavaScript Expressions

JavaScript can be embedded inside JSX.

```
function Student() {  
  
  const name = "Aditi";  
  
  const year = 2;  
  
  return (  
  
    <div>  
  
      <h2>Welcome {name}</h2>  
  
      <p>Year: {year}</p>  
  
    </div>  
  
  );  
  
}
```

Expressions can include:

```
{name}  
{year + 1}  
{student.department}  
{isLoggedIn ? "Logout" : "Login"}
```

📌 **Rule:** { } contains JavaScript expressions.

Slide 6 — Creating Reusable Components

Instead of repeating UI:

```
function StudentCard({ name, department, year }) {  
  
  return (  
  
    <div className="student-card">  
  
      <h3>{name}</h3>  
  
      <p>{department} — Year {year}</p>  
  
    </div>  
  
  );  
  
}
```

The same component can represent different students.

```
<StudentCard  
  name="Aditi"  
  department="CSE"  
  year={2}  
/>  
  
<StudentCard  
  name="Rahul"  
  department="ECE"  
  year={3}  
/>
```

One component → multiple UI instances

Slide 7 — What are Props?

Props = Properties

Props allow a parent component to send data to a child.

```
App
|
| props
▼
StudentCard
```

Think of props like function arguments.

Example:

```
<StudentCard
  name="Aditi"
  department="CSE"
  year={2}
/>
```

Inside the component:

```
function StudentCard({ name, department, year }) {

  // use props

}
```


Slide 8 — Props are Read-Only

A child should **not modify its props**.

```
function StudentCard({ name }) {
```

```
//
```

Slide 9 — Destructuring Props

Without destructuring:

```
function StudentCard(props) {  
  
  return (  
  
    <h3>{props.name}</h3>  
  
  );  
  
}
```

With destructuring:

```
function StudentCard({ name, department, year }) {  
  
  return (  
  
    <>  
  
      <h3>{name}</h3>  
  
      <p>{department}</p>  
  
      <p>Year {year}</p>  
  
    </>  
  
  );  
  
}
```

📌 **Recommended style:** Destructure props in the function parameter.

Slide 10 — children & Default Props

props.children

Allows content to be placed inside a component.

```
function Card({ children }) {  
  
  return (  
  
    <div className="card">  
  
      {children}  
  
    </div>  
  
  );  
  
}
```

Usage:

```
<Card>  
  
  <h2>Student Details</h2>  
  
  <p>Department: CSE</p>  
  
</Card>
```

Default values

```
function StudentCard({ name, year = 1 }) {  
  
  return (  
  
    <p>{name} - Year {year}</p>  
  
  );  
  
}
```

If year isn't supplied, React uses 1.

Slide 11 — Components + Props: Complete Example

```
function StudentCard({ name, department, year }) {  
  
  return (  
  
    <div>  
  
      <h3>{name}</h3>  
  
      <p>{department}</p>  
  
      <p>Year {year}</p>  
  
    </div>  
  
  );  
  
}  
  
function App() {  
  return (  
  
    <div>  
  
      <h1>Student Portal</h1>  
  
      <StudentCard  
        name="Aditi"  
        department="CSE"  
        year={2}  
      />  
  
      <StudentCard  
        name="Rahul"  
        department="ECE"  
        year={3}  
      />  
  
    </div>  
  
  );  
  
}
```

Concept flow:

Props

StudentCard

App

Dynamic UI

Slide 12 — Key Takeaways + Lab

Remember

Concept	Purpose
Component	Reusable UI
JSX	Describe UI
<code>{ }</code>	JavaScript expressions
Props	Pass data
children	Pass nested content
Destructuring	Cleaner prop access
Default value	Fallback prop value
One-way flow	Parent → Child